Proof Algebra

Luc Alexander Lüthi

April 8th, 2026

Contents

1 Introduction

The adversarial type theory lacked in that constraints could not be enforced for implementations, and more importantly that attributes could not be constructed, every implication invocation was flat and there was no state to reason about. This corrective note reformuates this system as an algebra of proof construction and inspection.

2 Instructions

Instructions take a proof as an argument, and return another more specified proof. Instructions are essentially just theorems.

$$\mathrm{id}\, x = x$$

The body of an instruction can be assertions and constructions. Implications are standard constructions, they take a proof which if true adds information to the proof being constructed.

$$\mathrm{f}\, a = a \to b$$

Assertions inspect proof objects to determine if they do or do not show a property. If they do not show the desired property the proof fails. $a \vdash b$ may be read a shows b.

$$\mathrm{f}\, a = a \vdash b\, b \to a$$

Instructions may be used as a construction, which nests the proof produced by the instruction in the calling instruction.

$$\mathrm{f}\, x = \mathrm{g}\, x \vdash (x \to y)$$

This algebra can be thought of as a constructive monotonic dependently typed proof algebra. Side effects may be given to instructions in order to make this algebra a computive dependently typed assembly language.

3 Environment

In order to adapt this to an adversarial model, a similar environment to the initial dependently typed model can be constructed. A universe consists of instructions with sufficiently intricate bodies. Minds can receive and emit signals in the environment, constructed as terms composed of other signals instructions. Defenderse prove and maintain a proof of certain target capability states, while attackers use those same generic theorems to prove undesirable capabilities. Capabilities are simply constructed true proofs which show some state is reachable given an instantiated input.